Sánchez Méndez
Cielali Itzel

Parcial 1 - Correcciones

0.1) Contesta con tus palabras, en unas tres oraciones. Puedes apoyarte en un ejemplo pequeño de código. En la vida gráfica de un objeto, ¿qué representa un atributo y qué representa un método? da un ejemplo de cada uno

El atributo representa la característica o el estado del objeto, y el método representa el comportamiento o lo que el objeto puede hacer; por ejemplo: el atributo puede ser el saldo de una cuenta bancaria y un método puede ser depositar. También podría ser el atributo velocidad de un coche y el método acelerar.

0.2) ¿Qué hace el método main y por qué se declara static en los programas que usamos en clase?

El método main funciona como el punto de entrada principal donde arranca la ejecución del programa. Se declara static para indicar que el método pertenece directamente a la clase, permitiendo que la JVM (Java Virtual Machine) lo llame sin necesidad de crear una instancia previa.

0.3) ¿Qué diferencia hay entre un atributo de instancia y uno static? explica qué comparten dos objetos de la misma clase

Un atributo static se define a nivel de clase y es accesible o compartido por todos, mientras que los atributos de instancia pertenecen individualmente a cada objeto creado. Por ejemplo, el color específico de cada objeto, donde cada uno guarda su propio valor de forma independiente

0.4) ¿Qué guarda una variable de tipo primitivo y qué guarda una

de referencia?. Da un ejemplo de cada cosa

Una variable primitiva almacena un valor numérico, booleano o de caracter de forma directa (int, double, boolean, char), una variable de referencia guarda una dirección en memoria que apunta hacia un objeto (String, int[])

4.5) ¿Qué hace javac con un archivo .java y qué hace la JVM con el resultado?.

javac toma el archivo de código fuente y lo traduce para producir un archivo intermedio de bytecode (.class) el cual es ejecutado por JVM. Por ejemplo: Si hacemos un Archivo.java, cuando ejecutemos el compilador obtendremos Archivo.class. Este archivo es el que se procesa para correr la aplicación sin requerir una traducción al lenguaje humano

R1) ¿Qué imprime?.

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido

```
int[] datos = {7, 5, 3};
int[] alias = datos;
int i=0;
while(i<3){
   System.out.println(alias[i]/2);
   i++;
}
alias = new int[] {9, 8, 6};
System.out.println("B"+ datos[0] + alias[0]);
double mitad= (double) (datos[1] /2);
System.out.println(mitad);
```

Línea/instrucción	i	datos	alias	Explicación	Salida
int[] datos={7,5,3};	—	{7,5,3}	—	Se crea el arreglo datos	—
int[] alias=datos;	—	{7,5,3}	{7,5,3}	alias apunta al mismo arreglo que datos	—
int i=0;	0	{7,5,3}	{7,5,3}	Se inicializa la variable de control i	—
while (i<3){ System.out.println(datos[i]/2);	0	{7,5,3}	{7,5,3}	0<3 es verdadero. datos[0]/2 = 7/2=3	3
i++	1	{7,5,3}	{7,5,3}	i (se incrementa 1)	—
while	1	{7,5,3}	{7,5,3}	1<3 es verdadero datos[1]/2 = 5/2=2	2
i++	2	{7,5,3}	{7,5,3}	i incrementa a 2	—
while	2	{7,5,3}	{7,5,3}	2<3 es verdadero datos[2]/2 = 3/2=1	1
i++	3	{7,5,3}	{7,5,3}	i incrementa a 3	—
alias=new int[]{9,8,6};	3	{7,5,3}	{9,8,6}	alias cambia y apunta a un nuevo arreglo	—
System.out.println("B"+datos[0] + alias[0]);	3	{7,5,3}	{9,8,6}	Concatenación: "B"+7(datos[0]+9 alias[0]) Se unen sin espacios	B79
double mitad=(double) (datos[1]/2);	3	{7,5,3}	{9,8,6}	División entera interna 5/2=2, el double lo convierte a valor 2.0	2.0

3
2
1
B79
2.0

← Respuesta

Rz a)
Cada inciso va en un main independiente. Indica no compila, escribe la salida tirena o funciona y da la causa en una frase. Si funciona,

```
int modo=1
Switch (modo) {
  case 1: System.out.println("LISTO");
  case 2: System.out.println("FIN");
}
```

Clasificación: funciona

Salida: LISTO //
FIN

Funciona y llega a esa salida por la regla de la caída en cascada (fall through). Como el case 1 no incluye break, la ejecución no se detiene al terminar de imprimir "LISTO", continúa de forma directa hacia el case 2 e imprime también "FIN".

Rz b)
Cada inciso va en un main independiente. Indica no compila, compila pero tirena o funciona y da la causa en una frase.

```
{ int nivel = 4; }
System.out.println(nivel);
```

Clasificación: no compila

El error es por un problema de ámbito (scope). La variable nivel está declarada dentro de un bloque delimitado por llaves {}. por lo tanto, solo existe y es visible dentro de ese entorno local. Al intentar invocarla fuera de las llaves en el System.out.println(nivel) java no lo reconoce y el código no compila

```
Public class Cargador {
  final int capacidad;
  int ocupadas;
  static int fabricados = 0;

  Public Cargador(int capacidad) {
    this.capacidad = capacidad;
    this.ocupadas = 0;
    fabricados++;
  }

  void colocar() {
    ocupadas++;
  }

  int consultarOcupadas() {
    return ocupadas;
  }
}
```

- Capacidad es final porque se fija al fabricarse y ya no cambia para ese objeto.
- fabricados es static porque es un contador global compartido por toda la clase para registrar el total de cargadores creados.

Ninguno de los dos necesita ser final y static al mismo tiempo.

→ Tipo: int para representar números enteros en capacidad, ranuras y contador.

→ Instancia/clase: capacidad y ocupadas son de instancia (cada objeto tiene los suyos), fabricados es de clase (static)

→ Final o mutable: Capacidad es final (inmutable tras crearse); ocupadas y fabricados son mutables

→ Constructor: recibe la capacidad por parámetro, inicializa el estado

y actualiza el contador global de fabricadas

→ Metodos: colocar() incrementa las ranuras ocupadas, y la consulta retorna el valor actual de ocupadas

R4)

El arreglo intentos contiene exactamente tres codigos enteros, en el orden en que se aprobaron. Devuelve el número del primer intento que coincide con clave (1, 2, 3). Si ninguno coincide, devuelve 0. Usa do-while y deja de revisar el acertar. El arreglo no es null. No lo modifiques.

static int intentoValido(int[] intentos, int clave)

Ejemplos (argumento → retorno):

{13, 42, 8} → 2 · {42, 8, 9}, 42 → 1 · {1, 2, 3} 42 → 0

Escribe el metodo completo. No imprimas ni leas el teclado

```
static int intentoValido(int[] intentos, int clave){
 int i=0;
 do{
   if(intentos[i] == clave){
     return i+1;
   }
   i++;
 }while(i<3);
   return 0;
}
```

- Ejemplo 1: Arreglo {13, 42, 8}, clave 42 → retorna 2

En la primera vuelta (i=0), el valor es 13, no coincide con 42.
En la segunda vuelta (i=1), el valor es 42, lo cual si coincide
el metodo devuelve i+1 (es decir, 1+1=2)

· Ejemplo 2: Arreglo {42, 8, 9}, clave 42 → retorna 1

En la primera vuelta (i=0), el valor es 42, encontrando la coincidencia de inmediato. El método devuelve i+1 (0+1=1) y detiene la búsqueda

· Ejemplo 3: {1, 2, 3}, clave 42 → retorna 0

El ciclo revisa las tres posiciones del arreglo (i=0, 1, 2) y ninguna coincide con el 42. Al terminar el ciclo do-while sin aciertos, el método retorna 0 tal como lo pedía la regla

R5)

Devuelva la acción del módulo con switch sobre Modo. Primero: Si bateria<10, devuelve "CARGAR", incluso con modo null. En otro caso, null devuelve "SIN MODO" antes del switch. El enum ya existe: escribe solo el método.

Regla o valor del recibido	Texto devuelto
APAGADO	"DETENER"
MEDIR	"LEER"
CALIBRAR	"AJUSTE"
PUERTA	"SIMULAR"

enum Modo {APAGADO, MEDIR, CALIBRAR, PRUEBA};

static String accion(Modo modo, int bateria)

```
enum Modo {APAGADO, MEDIR, CALIBRAR, PRUEBA}
static String accion(Modo modo, int bateria) {
  if (bateria<10) {
    return "CARGAR";
  }
  if (modo == null) {
    return "SIN MODO";
  }
```

```
Switch (modo) {
    case APAGADO:
        return "DETENER";
    case MEDIR:
        return "LEER";
    case CALIBRAR:
        return "AJUSTAR";
    case PRUEBA;
        return "SIMILAR";
    default:
        return " ";
    }
}
```

- Condición de bateria baja (bateria < 10)

Devuelve "CARGAR" porque el método valida primero si la bateria es menor a 10, si si retorna "CARGAR" sin importar el modo ni si es null.

- Condición de modo nulo (modo == null):

Devuelve "SIN MODO" porque si la bateria es de 10 o más pero el parámetro modo no tiene valor (null), se atrapa en esta condición antes de llegar al switch.

- modo APAGADO:

Devuelve "DETENER" porque el switch evalúa el enum y entra en el caso APAGADO, retornando el mismo texto

- modo MEDIR:

Devuelve "LEER" porque al coincidir con el valor MEDIR del enum el switch ejecuta la instrucción que retorna "LEER"

- Modo CALIBRAR:

devuelve AJUSTAR porque coincide con la opción CALIBRAR

- Modo PRUEBA:

Regresa SIMULAR porque el sector identifica el estado PRUEBA